RAILWAY RESERVATION SYSTEM

Ticket Booking and Cancellation using PL/SQL Stored Programs

(Procedures, Functions and Triggers in MySQL 8.0)

PL/SQL Programming

Final Project Report

Submitted by

KAVITA KUMARI

(UID: 26MCI10091)

Master of Computer Applications

IN

Artificial Intelligence and Machine Learning (MCA AI & ML)

UNIVERSITY INSTITUTE OF COMPUTING (UIC)

CHANDIGARH UNIVERSITY

Under the Supervision of

[Supervisor Name, Designation]

OCTOBER 2026

CERTIFICATE

This is to certify that the project report entitled "Railway Reservation System" has been prepared by Kavita Kumari (UID: 26MCI10091) as part of the academic work for the Master of Computer Applications (AI & ML) programme at the University Institute of Computing, Chandigarh University, for the subject PL/SQL Programming. The project shows how a train ticket booking and cancellation system can be built with tables, a trigger, a function and stored procedures, and how transactions keep the data correct.

Supervisor's Signature: _____

Name: _____

Date: _____

DECLARATION

I declare that this report describes the Railway Reservation System project that I have prepared for my coursework. The database design, the SQL code and the output screenshots included in this report are from the work done by me in MySQL Workbench. The names of passengers used in the tables are sample data only.

Student Signature: _____

Student Name: Kavita Kumari

ACKNOWLEDGEMENT

I would like to thank my faculty and my supervisor for guiding me during this project and for explaining the basics of stored programs and transactions in class. I am also thankful to the department for giving us the lab facilities. The official MySQL documentation helped me a lot while writing the trigger and the procedures, and my classmates helped with testing the booking and cancellation steps.

ABSTRACT

The Railway Reservation System is a small database project in which a passenger can be booked on a train seat and the booking can be cancelled later. The whole logic is written inside the database using stored programs. The project is made in MySQL 8.0 and run in MySQL Workbench, and it uses the same ideas that are taught in PL/SQL: procedures, functions, triggers, variables, conditions, exception handling and transactions.

There are four tables: train, passengers, seats and booking. The seats table keeps one row for every seat of every train and stores whether the seat is AVAILABLE or BOOKED. The procedure book_ticket checks the passenger, the train and the seat, locks the seat row, and then inserts a booking. A trigger named trg_booking_seat runs after the insert and marks the seat as BOOKED. The procedure cancel_ticket does the opposite: it changes the booking status to CANCELLED and makes the seat AVAILABLE again. A function, available_seats, returns the number of free seats of a train.

The project was tested with three trains of five seats each and three passengers. The ticket for passenger 1 on train 101, seat 1 was booked successfully, the seat table showed the seat as BOOKED, and the booking report showed the details of the ticket. The screenshots of these outputs are given in this report.

TABLE OF CONTENTS

Chapter	Title
1	Introduction
2	Problem Statement and Objectives
3	Scope, Limitations and Feasibility
4	Requirements and Tools
5	Database Design
6	PL/SQL Concepts Used in the Project
7	Implementation
8	Output Screenshots
9	Testing
10	Advantages, Limitations and Future Scope
11	Conclusion
12	Viva Questions and Answers
	References

LIST OF FIGURES

Figure	Title
Fig. 5.1	Relationship between the four tables
Fig. 8.1	Output of SELECT * FROM train
Fig. 8.2	Output of SELECT * FROM passengers
Fig. 8.3	Output of SELECT * FROM seats after booking seat 1
Fig. 8.4	Booking report using a join of three tables

1. INTRODUCTION

1.1 Background

Booking a train ticket looks easy for the passenger, but a lot of work goes on behind the screen. The system must check that the train exists, that the seat exists, and that nobody else has already taken the same seat. If two people try to book the same seat at the same moment, only one of them should get it. After a booking is made the seat must be shown as booked, and when the ticket is cancelled the seat must become free again for the next passenger.

All of these rules are about data, so they are best kept close to the data. If the rules are written inside the database as stored programs, every application that uses the database (a website, a mobile app or a counter software) follows the same rules automatically.

1.2 What is PL/SQL?

PL/SQL stands for Procedural Language extension to SQL. Normal SQL can only select, insert, update or delete rows, but PL/SQL adds programming features such as variables, IF conditions, loops, exception handling and named blocks of code. These named blocks are stored in the database as procedures, functions, triggers and packages, and they run on the database server.

PL/SQL is the language of Oracle Database. MySQL has its own stored program language which is very close to it. It also has DECLARE for variables, IF ... ELSEIF ... END IF for decisions, handlers for errors, and CREATE PROCEDURE, CREATE FUNCTION and CREATE TRIGGER for stored programs. In this project the programs are written in MySQL 8.0, so the syntax is that of MySQL, but the ideas are the same as the PL/SQL taught in the course.

1.3 About the Project

The Railway Reservation System has three trains (Shatabdi Express, Intercity Express and Himalayan Mail), each having five seats, and three sample passengers. The user can look at the trains, passengers and seats with simple SELECT statements, find the number of free seats with a function, book a ticket by calling one procedure, and cancel it by calling another.

1.4 Organisation of the Report

Chapter 2 gives the problem statement and objectives. Chapter 3 explains the scope and limits of the project. Chapter 4 lists the requirements and tools. Chapter 5 describes the database design. Chapter 6 explains the PL/SQL concepts used. Chapter 7 contains the implementation with the code. Chapter 8 shows the output screenshots. Chapter 9 gives the test cases. Chapter 10 discusses advantages, limitations and future scope, Chapter 11 is the conclusion and Chapter 12 has viva questions.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

In a manual or badly designed reservation system, the same seat can be given to two passengers, a cancelled seat may stay blocked, and the number of free seats may not match the real position. Keeping the booking rules only in the front-end program is risky, because a different program or a direct change in the database can break them. The problem is to design a railway reservation database in which the booking and cancellation rules are enforced inside the database itself, so that the seat information is always correct.

2.2 Objectives

- To design tables for trains, passengers, seats and bookings with proper primary keys, foreign keys and checks.
- To generate the seats of every train automatically from the total number of seats.
- To write a trigger that marks a seat as BOOKED as soon as a booking is inserted.
- To write a function that returns the number of available seats of a train.
- To write a procedure that books a ticket after checking the passenger, the train and the seat.
- To write a procedure that cancels a ticket and frees the seat.
- To use transactions, row locking and exception handling so that a failed booking leaves no half-finished data.
- To display booking reports and seat availability with join and group queries.

2.3 Proposed Solution

The solution keeps all rules in stored programs. The passenger and the train are checked first. Then the seat row is read with FOR UPDATE, which locks that row until the transaction ends, so another session has to wait. If the seat is free, a row is inserted in the booking table and the trigger changes the seat status. If anything goes wrong, an exit handler rolls the transaction back and sends the error message to the user.

3. SCOPE, LIMITATIONS AND FEASIBILITY

3.1 Scope

- Three trains with five seats each are stored; more trains and seats can be added with INSERT statements.
- A passenger can book one seat of one train at a time by calling book_ticket.
- A booking can be cancelled with cancel_ticket using the booking id.
- Reports are available for all bookings, active bookings, cancelled bookings and the seat availability of each train.
- The project can be used as the back-end of a web or desktop application.

3.2 Limitations

- There is no separate table for journey dates, so a seat belongs to a train and not to a train on a particular day.
- Seat classes (sleeper, AC) and fares are not stored.
- There is no user login, payment or ticket printing; it is only the database part.
- Only one seat is booked per call of the procedure.

3.3 Feasibility Study

Type	Assessment

Technical	MySQL 8.0 and MySQL Workbench are free and support stored procedures, functions, triggers and transactions.
Economic	No paid software is needed. Everything runs on a normal laptop.
Operational	The user only has to run simple CALL and SELECT statements, so it is easy to operate.
Schedule	The project is small and could be designed, coded and tested within a few weeks.

4. REQUIREMENTS AND TOOLS

4.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core processor, 1.5 GHz or higher
Memory (RAM)	4 GB or more
Storage	About 1 GB free space for MySQL Server and Workbench
Display	1366 x 768 or higher
Input devices	Keyboard and mouse

4.2 Software Requirements

Component	Requirement
Operating system	Windows 10 or 11
Database server	MySQL Server 8.0 (instance name: Local instance MySQL80)
Client tool	MySQL Workbench
Language	SQL with MySQL stored program language (PL/SQL style)
Script file	One .sql file containing the whole database, programs and queries

4.3 Why MySQL?

MySQL is free, easy to install and widely used. Version 8.0 supports CHECK constraints, which are used here for the seat count and the age, and it also supports the transaction-safe InnoDB engine with row-level locking. MySQL Workbench shows the result grid and the action output of every statement, which was useful for taking the screenshots in this report.

5. DATABASE DESIGN

5.1 Overview of the Tables

The database is named railway_reservation. It has four tables. The train table stores the trains, the passengers table stores the people who travel, the seats table stores every seat of every train, and the booking table connects a passenger with a seat. The relationship is shown in the figure below.

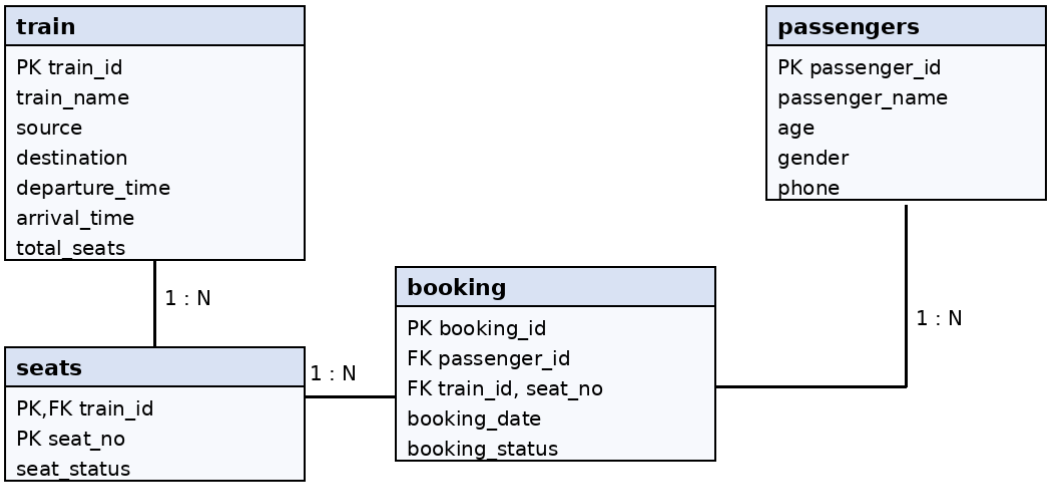


Fig. 5.1 Relationship between the four tables

5.2 Data Dictionary

Table 1: train

Column	Type	Description
train_id	INT, primary key	Unique number of the train
train_name	VARCHAR(100)	Name of the train
source	VARCHAR(80)	Starting station
destination	VARCHAR(80)	Ending station
departure_time	TIME	Time at which the train leaves
arrival_time	TIME	Time at which the train arrives
total_seats	INT, CHECK > 0	Number of seats in the train

Table 2: passengers

Column	Type	Description
passenger_id	INT, auto increment	Unique number of the passenger
passenger_name	VARCHAR(100)	Name of the passenger
age	INT, CHECK > 0	Age of the passenger
gender	VARCHAR(20)	Gender of the passenger
phone	VARCHAR(20)	Contact number

Table 3: seats

Column	Type	Description

train_id	INT, foreign key	Train to which the seat belongs (ON DELETE CASCADE)
seat_no	INT	Seat number inside the train
seat_status	ENUM('AVAILABLE','BOOKED')	Current status, default AVAILABLE

The primary key of this table is the pair (train_id, seat_no), because seat number 1 exists in every train but only once inside one train.

Table 4: booking

Column	Type	Description
booking_id	INT, auto increment	Unique number of the booking
passenger_id	INT, foreign key	Passenger who made the booking
train_id, seat_no	INT, foreign key	The seat that is booked (refers to seats)
booking_date	DATETIME	Date and time of booking, default current time
booking_status	ENUM('BOOKED','CANCELLED')	Status of the ticket, default BOOKED

5.3 Keys and Constraints

- Primary keys: train_id, passenger_id, booking_id and the pair (train_id, seat_no).
- Foreign keys: seats refers to train; booking refers to passengers and to seats.
- CHECK constraints: total_seats must be greater than 0 and age must be greater than 0.
- NOT NULL constraints on the important columns so that incomplete records are not stored.
- ENUM columns allow only the permitted status values.

5.4 Normalization

Every table has atomic values, so the design is in the first normal form. In every table each non-key column depends on the whole key, so it is in the second normal form; for example the train name depends only on train_id and not on the seat. No non-key column depends on another non-key column, so the tables are also in the third normal form. The passenger details are stored only once in the passengers table, and a booking refers to them by passenger_id, which avoids repetition.

6. PL/SQL CONCEPTS USED IN THE PROJECT

6.1 Stored Procedure

A stored procedure is a named block of code kept in the database and run with the CALL statement. It can take IN parameters, use variables and decide with IF conditions. In this project book_ticket and cancel_ticket are stored procedures. The advantage is that the program is written once and every user runs the same checked code.

6.2 Stored Function

A function is similar to a procedure but it returns one value and can be used inside a SELECT statement. The function available_seats(p_train_id) counts the AVAILABLE rows of a train and returns the number.

6.3 Trigger

A trigger is a program that the database runs by itself when an INSERT, UPDATE or DELETE happens on a table. The trigger trg_booking_seat is an AFTER INSERT trigger on booking. When a new booking with status BOOKED is inserted, it changes the matching seat to BOOKED. In this way the seat table stays in step with the booking table without any extra work from the user.

6.4 Variables and Conditions

Variables are declared with DECLARE, for example v_seat_status, v_passenger_count and v_train_count. IF, ELSEIF and ELSE are used to check the seat status and to decide whether to book or to raise an error.

6.5 Exception Handling

The procedures declare an EXIT HANDLER FOR SQLEXCEPTION. If any error happens, the handler rolls back the transaction and uses RESIGNAL to send the same error to the caller. User-defined errors are raised with SIGNAL SQLSTATE '45000' and a message such as 'Seat is already booked.'. This is like RAISE_APPLICATION_ERROR in Oracle PL/SQL.

6.6 Transactions and Locking

A transaction is a group of statements that must succeed together or fail together. START TRANSACTION begins it, COMMIT saves it and ROLLBACK cancels it. In book_ticket the seat row is read with SELECT ... FOR UPDATE. This locks the row, so if two users try to book the same seat, the second user waits until the first finishes and then sees the seat as BOOKED.

6.7 Comparison with Oracle PL/SQL

Feature	Oracle PL/SQL	MySQL (this project)
Procedure	CREATE PROCEDURE ... IS/AS	CREATE PROCEDURE ... BEGIN ... END
Variable	name datatype; in DECLARE	DECLARE name datatype;
Raise an error	RAISE_APPLICATION_ERRO R	SIGNAL SQLSTATE '45000'
Error handler	EXCEPTION WHEN ... THEN	DECLARE EXIT HANDLER FOR SQLEXCEPTION
Run a procedure	EXECUTE or CALL	CALL

7. IMPLEMENTATION

The complete work is written in one SQL script. It is divided into numbered sections so that it can be run step by step in MySQL Workbench. The main parts are explained below with the code.

7.1 Creating the Database and Cleaning Old Objects

The script first creates the database if it does not exist and selects it. Before creating anything it drops the old trigger, procedures, function and tables, so that the script can be run again and again without errors. The tables are dropped in the order booking, seats, passengers, train so that no foreign key blocks the deletion.

```
CREATE DATABASE IF NOT EXISTS railway_reservation;
USE railway_reservation;

DROP TRIGGER IF EXISTS trg_booking_seat;
DROP PROCEDURE IF EXISTS book_ticket;
DROP PROCEDURE IF EXISTS cancel_ticket;
DROP FUNCTION IF EXISTS available_seats;

DROP TABLE IF EXISTS booking;
DROP TABLE IF EXISTS seats;
DROP TABLE IF EXISTS passengers;
DROP TABLE IF EXISTS train;
```

7.2 Creating the Tables

The train and passengers tables are created first, then seats and booking. The seats table uses a composite primary key, and the booking table has a composite foreign key to it.

```
CREATE TABLE train (
    train_id INT PRIMARY KEY,
    train_name VARCHAR(100) NOT NULL,
    source VARCHAR(80) NOT NULL,
    destination VARCHAR(80) NOT NULL,
    departure_time TIME NOT NULL,
    arrival_time TIME NOT NULL,
    total_seats INT NOT NULL CHECK (total_seats > 0)
);

CREATE TABLE passengers (
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,
    passenger_name VARCHAR(100) NOT NULL,
    age INT NOT NULL CHECK (age > 0),
    gender VARCHAR(20),
    phone VARCHAR(20)
);
```

```
CREATE TABLE seats (
    train_id INT NOT NULL,
    seat_no INT NOT NULL,
    seat_status ENUM('AVAILABLE', 'BOOKED')
        NOT NULL DEFAULT 'AVAILABLE',
    PRIMARY KEY (train_id, seat_no),
    FOREIGN KEY (train_id) REFERENCES train(train_id)
        ON DELETE CASCADE
);
```

```
CREATE TABLE booking (
    booking_id INT PRIMARY KEY AUTO_INCREMENT,
    passenger_id INT NOT NULL,
    train_id INT NOT NULL,
    seat_no INT NOT NULL,
    booking_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    booking_status ENUM('BOOKED', 'CANCELLED')
        NOT NULL DEFAULT 'BOOKED',
    FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id),
    FOREIGN KEY (train_id, seat_no) REFERENCES seats(train_id, seat_no)
);
```

7.3 Inserting the Sample Data

Three trains are inserted, each with five seats. The seats are not typed by hand. Instead, the train table is cross joined with a small table of numbers from 1 to 10, and only the numbers up to total_seats are kept. In this way a train with five seats gets seats 1 to 5, and the script gives 15 seat rows for the three trains. Three passengers are then inserted.

```
INSERT INTO train VALUES
(101,'Shatabdi Express','Ludhiana','Delhi','06:00:00','11:30:00',5),
(102,'Intercity Express','Amritsar','Ludhiana','08:15:00','11:00:00',5),
(103,'Himalayan Mail','Delhi','Amritsar','19:00:00','05:30:00',5);

INSERT INTO seats (train_id, seat_no)
SELECT t.train_id, n.seat_no
FROM train t
CROSS JOIN (SELECT 1 AS seat_no UNION ALL SELECT 2 UNION ALL
            SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
            SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL
            SELECT 9 UNION ALL SELECT 10) n
WHERE n.seat_no <= t.total_seats;

INSERT INTO passengers (passenger_name, age, gender, phone) VALUES
('Aman Sharma', 24, 'Male', '9876500001'),
('Simran Kaur', 22, 'Female', '9876500002'),
('Rahul Verma', 30, 'Male', '9876500003');
```

7.4 Trigger: trg_booking_seat

The trigger runs after a row is inserted into booking. If the new booking has the status BOOKED, it updates the matching seat. DELIMITER $$ is used so that MySQL Workbench does not stop at the first semicolon inside the trigger body.

```
DELIMITER $$

CREATE TRIGGER trg_booking_seat
AFTER INSERT ON booking
FOR EACH ROW
BEGIN
    IF NEW.booking_status = 'BOOKED' THEN
        UPDATE seats
        SET seat_status = 'BOOKED'
```

```
        WHERE train_id = NEW.train_id
          AND seat_no = NEW.seat_no;
    END IF;
END$$
```

7.5 Function: available_seats

The function takes a train id, counts the seats of that train whose status is AVAILABLE, and returns the count. It only reads data, so it is declared READS SQL DATA.

```
CREATE FUNCTION available_seats(p_train_id INT)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_available INT DEFAULT 0;

    SELECT COUNT(*)
    INTO v_available
    FROM seats
    WHERE train_id = p_train_id
      AND seat_status = 'AVAILABLE';

    RETURN v_available;
END$$
```

7.6 Procedure: book_ticket

This is the main procedure. It takes the passenger id, the train id and the seat number. The steps are as follows.

- An exit handler is declared first, so that any error rolls back the transaction and is passed on with RESIGNAL.
- The transaction is started.
- The passenger and the train are counted; if the count is zero, an error 'Passenger does not exist.' or 'Train does not exist.' is raised.
- The seat status is set to NULL and then read with FOR UPDATE, which locks that row.
- If the status is still NULL the seat does not exist; if it is BOOKED the seat is already taken; otherwise the booking is inserted and the trigger marks the seat as booked.
- The transaction is committed and the message 'Ticket booked successfully.' is shown.

```
CREATE PROCEDURE book_ticket(
    IN p_passenger_id INT, IN p_train_id INT, IN p_seat_no INT)
BEGIN
    DECLARE v_seat_status VARCHAR(20) DEFAULT NULL;
    DECLARE v_passenger_count INT DEFAULT 0;
    DECLARE v_train_count INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
```

```
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT COUNT(*) INTO v_passenger_count
    FROM passengers WHERE passenger_id = p_passenger_id;
    IF v_passenger_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Passenger does not exist.';
    END IF;

    SELECT COUNT(*) INTO v_train_count
    FROM train WHERE train_id = p_train_id;
    IF v_train_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Train does not exist.';
    END IF;

    SET v_seat_status = NULL;
    SELECT seat_status INTO v_seat_status FROM seats
    WHERE train_id = p_train_id AND seat_no = p_seat_no
    FOR UPDATE;

    IF v_seat_status IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Seat does not exist.';
    ELSEIF v_seat_status = 'BOOKED' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Seat is already booked.';
    ELSE
        INSERT INTO booking
        (passenger_id, train_id, seat_no, booking_status)
        VALUES (p_passenger_id, p_train_id, p_seat_no, 'BOOKED');
    END IF;

    COMMIT;
    SELECT 'Ticket booked successfully.' AS message;
END$$
```

One small but important line is SET v_seat_status = NULL before the SELECT. If the seat number does not exist, the SELECT finds no row and the variable is not changed. By setting it to NULL first, the program can tell that no seat was found.

7.7 Procedure: cancel_ticket

This procedure takes a booking id. It reads the train id, seat number and status of that booking with FOR UPDATE. If the booking is not found, or it is already cancelled, an error is raised. Otherwise the booking status is changed to CANCELLED and the seat is changed back to AVAILABLE. The whole work is in one transaction.

```
CREATE PROCEDURE cancel_ticket(IN p_booking_id INT)
BEGIN
    DECLARE v_train_id INT DEFAULT NULL;
    DECLARE v_seat_no INT DEFAULT NULL;
```

```
    DECLARE v_status VARCHAR(20) DEFAULT NULL;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT train_id, seat_no, booking_status
    INTO v_train_id, v_seat_no, v_status
    FROM booking WHERE booking_id = p_booking_id
    FOR UPDATE;

    IF v_status IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking does not exist.';
    ELSEIF v_status = 'CANCELLED' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Ticket is already cancelled.';
    ELSE
        UPDATE booking SET booking_status = 'CANCELLED'
        WHERE booking_id = p_booking_id;

        UPDATE seats SET seat_status = 'AVAILABLE'
        WHERE train_id = v_train_id AND seat_no = v_seat_no;
    END IF;

    COMMIT;
    SELECT 'Ticket cancelled successfully.' AS message;
END$$

DELIMITER ;
```

7.8 Report Queries

After the programs are created, the script has queries to display the tables, to check the available seats of train 101, to show all bookings, and to show two reports. The booking report joins the booking, passengers and train tables. The availability report groups the seats by train and counts available and booked seats with SUM on a condition.

8. OUTPUT SCREENSHOTS

The screenshots below were taken in MySQL Workbench (Local instance MySQL80) after running the script. The query is selected in the editor, the result is shown in the result grid, and the Action Output panel at the bottom shows that every statement ran successfully with a green tick.

8.1 Train Table

The statement SELECT * FROM train; shows the three trains: Shatabdi Express (101) from Ludhiana to Delhi, Intercity Express (102) from Amritsar to Ludhiana, and Himalayan Mail (103) from Delhi to Amritsar. Each has 5 seats. The action output also shows the earlier statements, including CALL book_ticket(1, 101, 1), which returned one row.

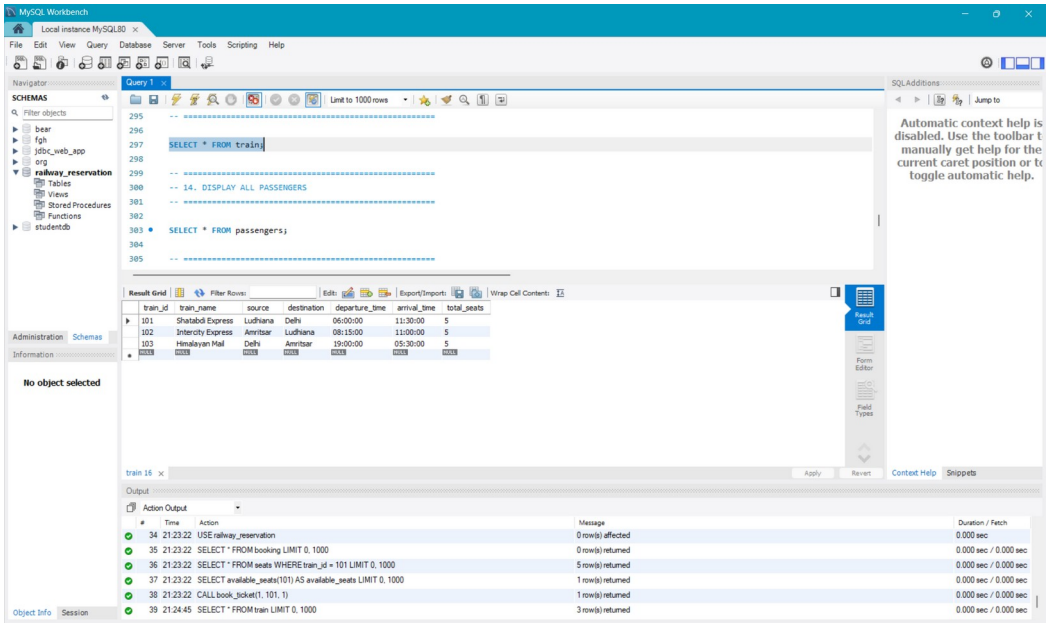


*Fig. 8.1 Output of SELECT * FROM train*

8.2 Passengers Table

The passengers table has three rows: Aman Sharma (24, Male), Simran Kaur (22, Female) and Rahul Verma (30, Male), with passenger ids 1, 2 and 3 generated by AUTO_INCREMENT.

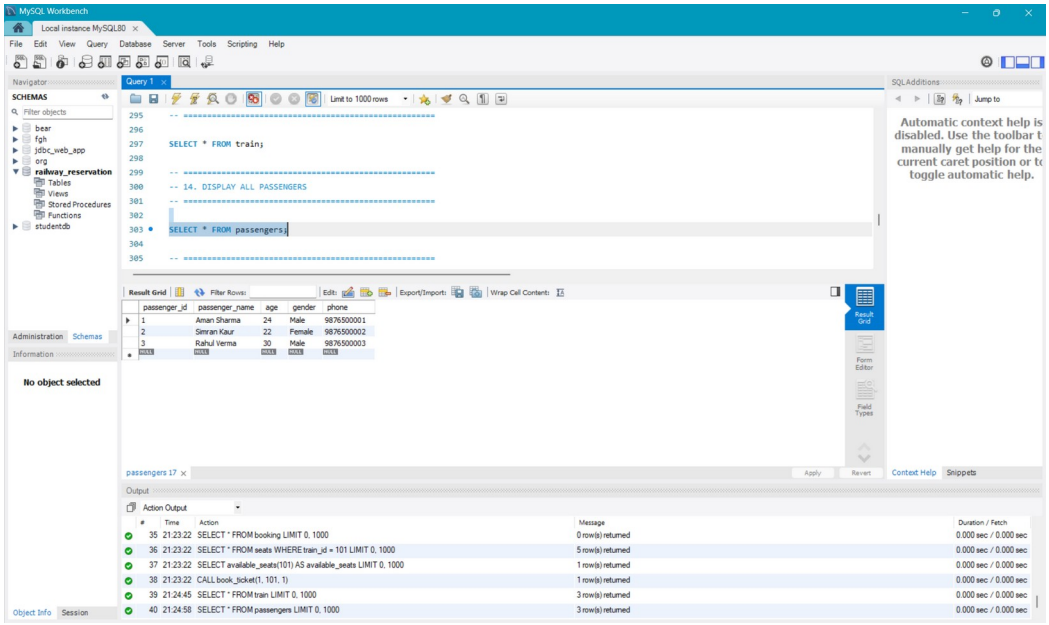


*Fig. 8.2 Output of SELECT * FROM passengers*

8.3 Seats Table after Booking

The seats query returns 15 rows, which are 5 seats for each of the three trains. In the result, seat 1 of train 101 has the status BOOKED, while all the other seats are AVAILABLE. This change was not made by hand; it was made by the trigger trg_booking_seat when book_ticket(1, 101, 1) inserted the booking.

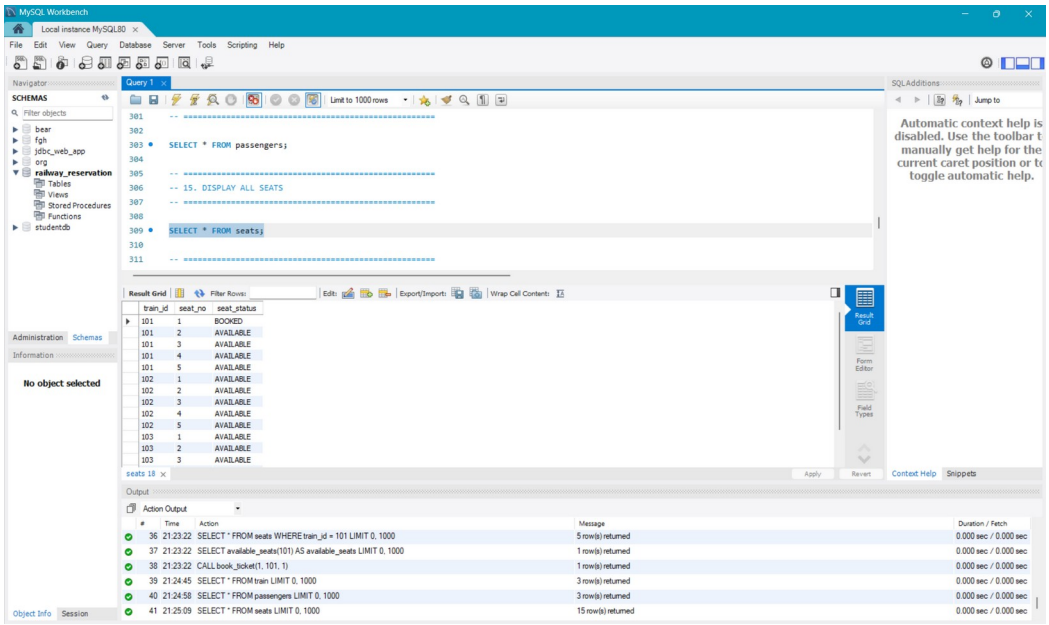


*Fig. 8.3 Output of SELECT * FROM seats after booking seat 1 of train 101*

8.4 Booking Report

The booking report joins three tables. It shows that booking 1 belongs to Aman Sharma on the Shatabdi Express from Ludhiana to Delhi, for seat 1, booked on 2026-10-27 at 21:23:22, with the status BOOKED. This matches the seat status seen in Fig. 8.3.

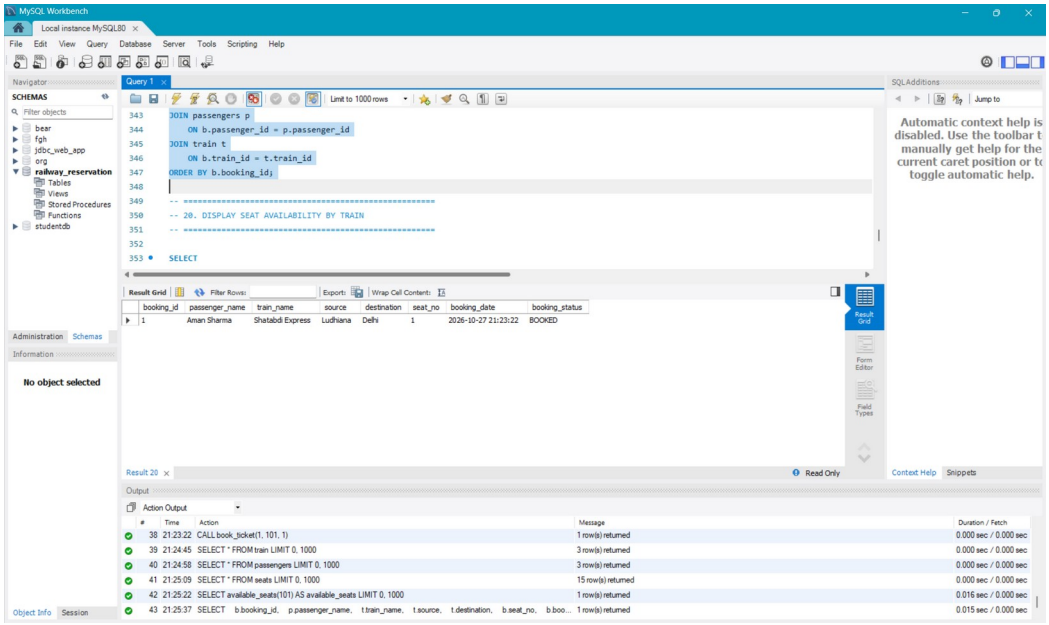


Fig. 8.4 Booking report using a join of three tables

8.5 Summary of the Outputs

Statement	Result seen
SELECT * FROM train	3 rows returned
SELECT * FROM passengers	3 rows returned
SELECT * FROM seats	15 rows returned; seat 1 of train 101 is BOOKED
CALL book_ticket(1, 101, 1)	Executed successfully and returned 1 row (the success message)
Booking report (3-table join)	1 row: booking 1, Aman Sharma, Shatabdi Express, seat 1, BOOKED

9. TESTING

9.1 Test Cases

The test cases below cover the normal flow and the error cases written in the procedures. The first test was run and its output is shown in Chapter 8. For the other cases the table gives the expected result, which follows from the code of the procedures; they can be run in the same way in MySQL Workbench.

No.	Statement	Expected output	Seen in	Status
1	CALL book_ticket(1, 101, 1);	Ticket booked successfully; seat 1 becomes BOOKED	Fig. 8.3, 8.4	Run
2	CALL book_ticket(2, 101, 1);	Error: Seat is already booked.	-	To run
3	CALL book_ticket(9, 101, 2);	Error: Passenger does not exist.	-	To run
4	CALL book_ticket(1, 999, 1);	Error: Train does not exist.	-	To run
5	CALL book_ticket(1, 101, 9);	Error: Seat does not exist.	-	To run

6	SELECT available_seats(101);	5 before booking and 4 after booking	-	5 seen before booking
7	CALL cancel_ticket(1);	Ticket cancelled successfully; seat 1 becomes AVAILABLE	-	To run
8	CALL cancel_ticket(1); again	Error: Ticket is already cancelled.	-	To run
9	CALL cancel_ticket(50);	Error: Booking does not exist.	-	To run

9.2 Observations

- After a successful booking, the booking table and the seats table agree: the booking shows BOOKED and the seat shows BOOKED.
- Because the seat row is locked with FOR UPDATE, two sessions cannot book the same seat together.
- If any step fails, the exit handler rolls the transaction back, so no half booking is stored.
- The trigger removes the need to update the seat manually, which avoids mistakes.

9.3 Problems Faced

While writing the project, the main problem was the semicolon inside the procedures. MySQL Workbench treats a semicolon as the end of a statement, so the code of a trigger or a procedure was cut in the middle. This was solved by changing the delimiter to $$ before the programs and changing it back to a semicolon after them. Another problem was a seat number that does not exist: the SELECT INTO found no row and the old value of the variable remained, so the variable is set to NULL just before the SELECT. Dropping the old objects at the start of the script also helped, because without it the script failed when it was run for the second time.

10. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

10.1 Advantages

- The booking rules are stored in the database, so every application follows the same rules.
- Transactions and row locking prevent double booking of a seat.
- Errors are clear, with messages such as 'Seat is already booked.'.
- The trigger keeps the seat status correct automatically.
- The script can be run again from the start without errors.
- Reports are easy to produce with ordinary SELECT queries.

10.2 Limitations

- Seats are linked to a train and not to a journey date.
- There is no fare, class of travel or payment.
- A passenger can book only one seat per procedure call and there is no waiting list.
- There is no front-end screen; the system is used from SQL statements.

10.3 Future Scope

- Add a journey table with dates so that the same train can be booked for different days.
- Add coach classes, fares and a payment table.
- Add a waiting list and automatic confirmation when a ticket is cancelled.
- Add a user table with login and roles for the admin and the passenger.
- Use a cursor in a procedure to print the passenger list of a train.
- Connect the database with a web application written in Java, Python or PHP.

11. CONCLUSION

The Railway Reservation System shows how the main rules of ticket booking can be placed inside the database with stored programs. The tables are connected with keys and constraints, the seats are generated automatically, the trigger keeps the seat status up to date, the function gives the number of free seats, and the two procedures book and cancel tickets safely inside transactions.

The output screenshots prove that the trains, passengers and seats are created correctly, that book_ticket booked seat 1 of the Shatabdi Express for Aman Sharma, and that the trigger changed the seat to BOOKED. Through this project I understood the use of procedures, functions, triggers, exception handling, SIGNAL and RESIGNAL, and row locking. The project can be extended into a complete reservation application by adding journey dates, fares and a front-end.

12. VIVA QUESTIONS AND ANSWERS

Q1. What is PL/SQL?
Ans. It is a procedural extension of SQL that adds variables, conditions, loops, exception handling and stored programs such as procedures, functions and triggers.

Q2. Which database did you use?
Ans. MySQL 8.0 with MySQL Workbench. Its stored program language is very close to PL/SQL.

Q3. What is a stored procedure?
Ans. A named block of code stored in the database which is run with the CALL statement and can take parameters.

Q4. How is a function different from a procedure?
Ans. A function returns a value and can be used inside a SELECT statement; a procedure is run with CALL and need not return a value.

Q5. What is a trigger? Which one did you use?
Ans. A trigger runs automatically on INSERT, UPDATE or DELETE. I used trg_booking_seat, which marks a seat as BOOKED after a booking is inserted.

Q6. Why is DELIMITER $$ used?
Ans. So that the semicolons inside a procedure or trigger body do not end the CREATE statement early.

Q7. What does FOR UPDATE do?
Ans. It locks the selected row until the transaction ends, so another session cannot change or book the same seat meanwhile.

Q8. What is the use of START TRANSACTION, COMMIT and ROLLBACK?

Ans. START TRANSACTION begins a group of statements, COMMIT saves them permanently and ROLLBACK undoes them if there is an error.

Q9. What is an EXIT HANDLER?

Ans. It is a block that runs when an error occurs. In this project it does a ROLLBACK and then RESIGNAL to pass the error on.

Q10. Why does the seats table have a composite primary key?

Ans. Seat numbers repeat in different trains, so a seat is identified only by the pair (train_id, seat_no).

Q11. What is ON DELETE CASCADE in seats?

Ans. If a train is deleted, all its seats are deleted automatically.

Q12. How are seats created automatically?

Ans. By cross joining the train table with numbers 1 to 10 and keeping the numbers that are less than or equal to total_seats.

Q13. What happens when a booking is cancelled?

Ans. The booking status is changed to CANCELLED and the seat status is changed back to AVAILABLE.

Q14. What is the output of available_seats(101) before any booking?

Ans. 5, because all five seats of train 101 are AVAILABLE.

REFERENCES

[1] MySQL 8.0 Reference Manual: Stored Objects (procedures, functions, triggers), https://dev.mysql.com/doc/refman/8.0/en/stored-objects.html

[2] MySQL 8.0 Reference Manual: SIGNAL and RESIGNAL statements, https://dev.mysql.com/doc/refman/8.0/en/signal.html

[3] MySQL 8.0 Reference Manual: Locking reads (SELECT ... FOR UPDATE), https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html

[4] MySQL Workbench Manual, https://dev.mysql.com/doc/workbench/en/

[5] Steven Feuerstein and Bill Pribyl, Oracle PL/SQL Programming, O'Reilly Media.

[6] R. Elmasri and S. Navathe, Fundamentals of Database Systems, Pearson.